

Cairo University Racing Team

Generative AI Technical Questions

1. Explain the difference between generative and discriminative models. Give one example of each.

Generative models learn how the data is distributed and can create new data similar to what they learned. For example, GPT can generate text. Discriminative models learn how to distinguish between classes or predict a label. For example, a spam classifier can predict whether an email is spam or not.

2. What is the purpose of transformers in modern AI? How does it work briefly?

Transformers are used to understand and generate sequences such as text. They use attention to look at the important parts of the input and understand relationships between words, even when they are far apart. A transformer is mainly built from attention layers and feed-forward layers.

3. Describe the difference between supervised, unsupervised, and reinforcement learning.

Supervised learning uses labeled data, so the model learns from inputs with known answers. Unsupervised learning uses data without labels and tries to find patterns or groups. Reinforcement learning learns by interacting with an environment and receiving rewards or penalties for its actions.

4. What are tokens in the context of Large Language Models? Why are they important?

Tokens are the basic fragments of text that an LLM processes. A token can be a whole word, part of a word, or sometimes a punctuation mark. They are important because models process text as tokens, and token limits affect how much text can fit into the context window and how much an API request costs.

5. Explain the concept of attention in neural networks. How does it improve model performance?

Attention lets a model decide which parts of the input are more important when producing an output. In language, it helps the model connect words that are related even if they are far apart. This makes it better at understanding context and relationships in a sentence.

6. What is prompt engineering, and why is it crucial for Gen AI applications? State the different types of prompts used.

Prompt engineering is really just being deliberate about how you phrase instructions to an LLM so you get consistent, accurate results instead of a shot in the dark. A few common patterns:

- Zero-shot — you just ask the question directly, no examples given.
- Few-shot — you show a couple of example input/output pairs first, so the model picks up the pattern.
- Role-based — you give the model a persona to play, e.g. "act as a suspension design engineer."
- Structured prompting — you force the output into a specific shape, like JSON, Markdown, or a bullet list, so it's easy to parse downstream.

7. Explain how RAG (Retrieval-Augmented Generation) works.

RAG is a way to give an LLM access to outside knowledge without retraining it. You take your documents, break them into chunks, and turn each chunk into a vector embedding stored in a database. When someone asks a question, the system searches that database for the most relevant chunks and stuffs them into the prompt — so the model answers using real, grounded information instead of just what it memorized during training.

8. What is embedding in AI, and why is it important for retrieval tasks?

An embedding is a numerical vector that represents the meaning or features of data such as text. Similar pieces of information have similar vectors. This makes embeddings useful for semantic search, because we can find documents that are related to a question even when they do not use exactly the same words.

9. How does temperature affect the output of a language generation model?

Temperature controls how random the model's next-token choices are. A low temperature usually gives more predictable and focused answers. A higher temperature allows more variety and creativity, but it can also make answers less consistent.

10. What is chain-of-thought prompting, and why can it improve reasoning in LLMs?

Chain-of-thought prompting encourages a model to work through a problem step by step instead of jumping directly to an answer. This can help with tasks that require multiple reasoning steps, such as calculations or logical problems. In practice, systems should avoid exposing private internal reasoning and instead provide concise explanations or results when appropriate.

11. What is the difference between RAG and fine-tuning? Give a use case for each.

RAG gives the model external information at query time, so it is useful when the knowledge Both extend what a base LLM can do, but in different ways.

- RAG hands the model an "open textbook" at the moment you ask a question — useful for up-to-date or private facts, like pulling answers straight from CURT's vehicle design logs.
- Fine-tuning actually retrains the model's weights, so it's better suited to teaching a consistent style, tone, or format — for example, training a model to always output telemetry code in your team's exact syntax.

12. Explain the possible improvements that can be applied to a Naive RAG system, both pre-retrieval and post-retrieval.

Before retrieval, we can improve document cleaning, chunking, metadata, query rewriting, and the embedding model. After retrieval, we can rerank the retrieved chunks, remove irrelevant results, combine useful context, and limit the context sent to the LLM. These steps can improve both retrieval quality and the final answer.

13. What factors should you consider when choosing an LLM for an application? Consider accuracy, latency, cost, context window, privacy, and reasoning ability.

I would compare the model's accuracy and reasoning ability for the actual task, its response speed, API cost, context-window size, and privacy requirements. I would also consider tool/function-calling support, reliability, and how easy it is to integrate. The best choice depends on the application's requirements rather than only the model's benchmark scores.

14. How can you evaluate whether an LLM-generated answer is grounded in the provided documents?

I would check whether the answer is supported by the retrieved documents and whether its claims can be traced back to the provided sources. Useful evaluation methods include human review, citation checking, and test questions with known answers. For a RAG system, I would separately evaluate retrieval quality and whether the generated answer stays within the retrieved evidence.

15. How do you think computer vision could be integrated with Generative AI?

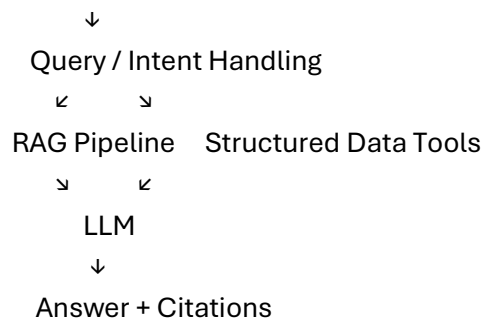
Computer vision can provide information from images or videos, while a generative model can explain or use that information. For example, a CURT assistant could analyze an image of a car part and then answer questions about the part or retrieve related documentation. Multimodal models can combine image and text understanding in one system.

16. Design an AI assistant for CURT.

The assistant should let members from different CURT teams ask questions about their documents and structured data. It would combine document retrieval, structured-data tools, an LLM, access control, and source citations.

16.a. System architecture

User → Web UI → Authentication/Authorization → Assistant Backend



The document side would contain file parsing, chunking, embeddings, and a vector database. Structured data would be accessed through controlled backend functions. Authentication and authorization would make sure users only retrieve information they are allowed to access.

16.b. Main challenges

- Different file formats need different parsing methods.
- Data may be incomplete, inconsistent, or noisy.
- Large documents cannot all fit into an LLM context window, so chunking is needed.
- Retrieval may return irrelevant or incomplete information.
- The LLM may generate an answer that is not supported by the retrieved data.

16.c. Possible solutions

First, use suitable parsers and a preprocessing pipeline to clean and normalize the documents before indexing them. Second, improve retrieval with good chunking, metadata, embeddings, and reranking. I would also require citations and restrict the LLM to retrieved information and approved backend tools.

16.d. Evaluation

Create a test set of real CURT questions with expected answers and sources. Measure whether the correct documents are retrieved, whether answers are supported by those documents, and whether citations are correct. I would also test access control, different file types, ambiguous questions, and cases where the answer is not available.